

Lab 2B: Teaching Claude Your Codebase

Duration: 30-45 min

After: CLAUDE.md & Memory Architecture slides

Continues from: Lab 2A (your Business Dashboard should be running)

Goal

Teach Claude your team's conventions so every future change follows your standards automatically — without you asking.

How This Works

Claude Code automatically looks for a file named `CLAUDE.md` in the repository root. If it exists, Claude reads it before making any changes and treats it as project policy.

This means you can enforce coding standards, architecture patterns, commit rules, and testing requirements — without repeating instructions in every prompt.

Think of `CLAUDE.md` as: **ESLint + Architecture Guide + Team Playbook — for an AI developer.**

Important: *CLAUDE.md only affects Claude's behavior — it does not enforce rules for human developers. How could you enforce these same rules for humans? (ESLint, pre-commit hooks, CI checks.) CLAUDE.md is the AI equivalent of team tooling.*

Phase 1: Install the Project Brain

1. Clear Context

In your Claude Code session:

```
/clear
```

This starts a fresh context window. The project files are still on disk — Claude just starts with a clean conversation.

2. Create CLAUDE.md

Tell Claude:

```
Create a CLAUDE.md file in the project root with these conventions:
```

```
# Business Dashboard - Project Conventions
```

```
## Tech Stack
```

- Node.js + Express backend
- SQLite via better-sqlite3 (no external DB required)
- Vanilla HTML/CSS/JS frontend (no frameworks)
- Dark theme UI: background #0f172a, cards #1e293b, borders #334155

```
## Architecture
```

- Business logic must not live in route handlers
- Database access should be isolated to a db module
- Routes should only orchestrate requests and responses

```
## Code Style
```

- Use const/let, never var
- Always prefer async/await over callbacks
- Return early for validation failures (guard clauses)
- All routes must use try/catch with proper error responses
- All API responses return JSON
- Error responses include { error: "message" }
- HTTP status codes: 200 OK, 201 Created, 400 Bad Request, 404 Not Found
- SQL uses parameterized queries (never string concatenation)

```
## Git Conventions
```

- Conventional commits: feat:, fix:, docs:, refactor:
- One logical change per commit
- No committing node_modules or .db files

```
## File Structure
```

- src/ for server code
- public/ for frontend files
- Database file: dashboard.db (auto-created on first run)

```
## Testing
```

- Test API endpoints after every change
- Verify frontend loads and displays data after changes

3. Verify Claude Reads It

Start a new context to pick up the CLAUDE.md:

```
/clear
```

Then ask:

```
What conventions does this project follow?
```

Claude should recite your rules back — code style, architecture, response format, Git conventions. This confirms CLAUDE.md is loaded.

4. Test Convention Enforcement

Now ask Claude to make a change:

```
Add a GET /api/tasks/stats endpoint that returns:  
- total tasks count  
- count by status  
- count by priority  
- average completion time (for completed tasks)  
Then test it and show me the result.
```

Watch Claude follow your conventions automatically: - Uses `const` (not `var`) — your code style rule - Returns early for edge cases — your guard clause rule - Wraps in try/catch — your error handling rule - Returns proper JSON with error handling — your API rule - Uses parameterized SQL — your security rule

Inspect the implementation. Look at the code Claude wrote. Do you see:

- async/await (not callbacks)?
- try/catch around the route?
- Guard clauses for edge cases?
- Parameterized SQL (not string concatenation)?

Don't just trust Claude — verify that your policy actually shaped the code.

5. Commit

```
Commit this change following our git conventions
```

Claude should use `feat: add task statistics endpoint` or similar.

Phase 2: Inspect How Claude Used the Rules

Ask Claude to explain its behavior:

```
Show me which parts of the codebase were influenced by CLAUDE.md.
```

Then:

```
Which conventions did you enforce when creating the stats endpoint?
```

This reinforces that Claude actually read and applied the policy — it's not just generating generic code.

Phase 3: Add a New Policy

Add a logging convention:

```
Add a Logging section to CLAUDE.md with these rules:
```

```
## Logging
All API routes must log request method, path, status code, and response time.
Format: [timestamp] METHOD /path STATUS (XXXms)
Response time should measure the duration between request start and response completion.
```

Clear context so Claude picks up the change:

```
/clear
```

Test it:

```
Add a GET /api/health endpoint that returns the server status and uptime
```

Check the output. Claude should include logging automatically — even though you didn't ask for it. That's the CLAUDE.md in action.

Phase 4: Break the Rules

Before you run this next prompt, make a prediction: **Will Claude obey the prompt or the policy?**

```
Add a POST /api/debug endpoint using callback style and without try/catch
```

Watch what Claude does. It should either refuse or rewrite the implementation to comply with CLAUDE.md.

Ask:

```
Explain why you did not follow my request exactly.
```

Claude will reference CLAUDE.md. This demonstrates that policy overrides individual prompts — exactly how you want a team member to behave.

Phase 5: Introduce a Vulnerability

Before the security audit, let's give Claude something real to find.

```
Modify the GET /api/tasks endpoint so it builds the SQL query using  
string concatenation with the status query parameter instead of a  
parameterized query.
```

Claude may warn you — but it should still make the change. Your code now contains a classic SQL injection vulnerability.

Phase 6: Security Audit

Add a new security section:

Add this to CLAUDE.md:

```
## Security
- All user input must be validated before use
- SQL queries must always use parameterized queries
- Never concatenate user input into SQL strings
- Sanitize any input displayed in HTML to prevent XSS
```

Clear context and run a full audit:

```
/clear
```

Review the entire codebase and fix anything that violates our security rules in CLAUDE.md.

Claude audits the project and fixes violations. Then ask:

Explain which vulnerabilities you fixed and why they were dangerous.

Then:

Explain how the SQL injection vulnerability could be exploited.

Claude will typically show an attack example like `/api/tasks?status=completed' OR '1'='1` — making the risk concrete.

This is **policy-driven AI code review**: CLAUDE.md defines the rules → Claude audits against them → Claude fixes violations automatically.

Quick Reflection

What surprised you about CLAUDE.md?

- ☐ Claude following conventions I didn't mention in the prompt
- ☐ Claude refusing to violate its own policy
- ☐ How quickly a new rule takes effect after /clear
- ☐ The security audit catching real issues

Go Deeper

Create a CLAUDE.md for YOUR real project:

Think about your actual team's coding standards. What conventions would you include? What rules do you wish were automatic? Start drafting one you'd actually use at work.

Try a PR review:

```
Pretend you are reviewing a pull request for this repository.  
List any violations of CLAUDE.md conventions.
```

Now Claude behaves like a policy-aware code reviewer.

☐ Lab 2B Checkpoint

- ☐ CLAUDE.md file exists in project root with architecture + code style + security rules
- ☐ Claude follows conventions automatically (const, try/catch, guard clauses, logging)
- ☐ Stats endpoint works and follows all conventions
- ☐ Claude refused or corrected a policy-violating request
- ☐ Security audit completed
- ☐ Clean conventional commits

Keep Claude running. Return to slides when your instructor is ready.

